REVISION CHEAT SHEET:

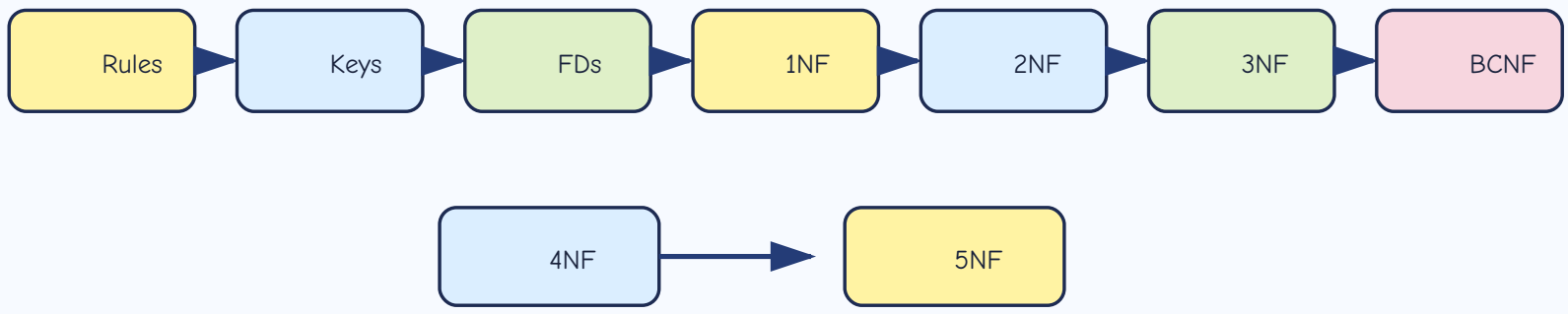
SQL Server Database Normalization

INTERVIEW-READY NOTES - NORMAL FORMS - SQL - DIAGRAMS

DIAGRAM

Normalization Roadmap

From business rules to advanced normal forms



Fundamentals first; specialized normal forms only when the business rules require them.

Topic & Subtopic Index

Study order: foundations -> dependencies -> 1NF -> 2NF -> 3NF -> BCNF -> 4NF -> 5NF -> decomposition quality -> practical SQL Server design -> revision.

1. Foundations of Normalization

- What Is Database Normalization?
- Why Normalize a Database?
- Data Redundancy
- Duplicate Data Does Not Always Mean Bad Design
- Business Facts Are the Key to Normalization
- Database Anomalies

2. Keys and Functional Dependencies

- Keys You Need Before Learning Normal Forms
- Superkey
- Candidate Key
- Primary Key
- Composite Key
- Functional Dependency — Simple Definition
- Full Functional Dependency
- Partial Functional Dependency
- Transitive Functional Dependency

3. UNF and First Normal Form

- Normalization Starting Point — UNF
- UNF Example Using FundraisingDb
- Another UNF Example — Repeating Columns
- Atomic vs Non-Atomic Values
- UNF to 1NF — Three Frames
- First Normal Form — 1NF
- 1NF Example
- Important 1NF Rule

4. Second Normal Form

- Second Normal Form — 2NF
- Real Fundraising Example for 2NF
- Analyze Every Column Carefully
- 2NF Problem Diagram
- Convert to 2NF
- 2NF Three-Frame Diagram
- Why 2NF Matters

5. Third Normal Form

- Third Normal Form — 3NF
- Real Fundraising Example for 3NF
- Constituent Transitive Dependency
- Campaign Transitive Dependency
- Convert to 3NF
- 3NF Three-Frame Diagram
- Classic 3NF Memory Rule
- 1NF vs 2NF vs 3NF

6. BCNF, 4NF, and 5NF

- Boyce-Codd Normal Form — BCNF
- Simple BCNF Definition
- BCNF Learning Example
- BCNF Decomposition
- 3NF vs BCNF
- Fourth Normal Form — 4NF
- Correct 4NF Design
- Fifth Normal Form — 5NF
- Simple 5NF Idea
- Fifth Normal Form Definition

7. Decomposition Quality

- Lossless Decomposition
- Lossless Example Using Constituent and Gift
- Lossy Decomposition
- Lossy Example Using Gift, Campaign, and Fund
- Lossless vs Lossy
- Dependency Preservation
- Lossless vs Dependency Preservation
- Ideal Decomposition

8. Integrity and Practical SQL Server Design

- Normalization and Referential Integrity
- Normalization Does Not Replace Constraints
- Practical SQL Server Example
- Normalization of Constituent Addresses
- Why AddressType Belongs in ConstituentAddress
- Normalization of Gift and Fund

9. Denormalization and Performance

- Denormalization
- Fundraising Denormalization Example
- Denormalization Risk
- Controlled Denormalization
- When to Consider Denormalization
- When Not to Denormalize
- Normalization vs Denormalization
- Normalize First, Optimize Second
- Normalization vs Performance

10. Practical Method and Mistakes

- Choosing the Appropriate Normal Form
- Typical Fundraising OLTP Target
- Normalization Is Not "More Tables Is Better"
- Status and Type Columns
- Practical Normalization Process
- Three-Frame Method for Normalization Questions
- Common Normalization Mistakes
- Master Normalization Example

11. Revision and Interview Review

- Complete Normal-Form Memory Table
- Dependency Memory Table
- 25 Interview Questions
- Final Revision Diagram
- Final Easy Memory Rule

Database normalization is a **database-design process** used to decide where each business fact should be stored.

The main purpose is not simply:

EXAMPLE / FLOW

```
Create more tables
```

The real purpose is:

EXAMPLE / FLOW

```
Store each business fact
in the correct place
so that the same fact
does not need to be maintained
unnecessarily in many places.
```

Throughout these notes we will use the same fundraising domain:

EXAMPLE / FLOW

```
core.Tenant

core.AppUser

constituent.Constituent

constituent.Address

constituent.ConstituentAddress

fundraising.Campaign

fundraising.Fund
```

```
fundraising.Gift

fundraising.GiftAllocation

engagement.Interaction

engagement.Task
```

These are the main business tables in your **FundraisingDb**.

The most important fundraising flow is:

EXAMPLE / FLOW

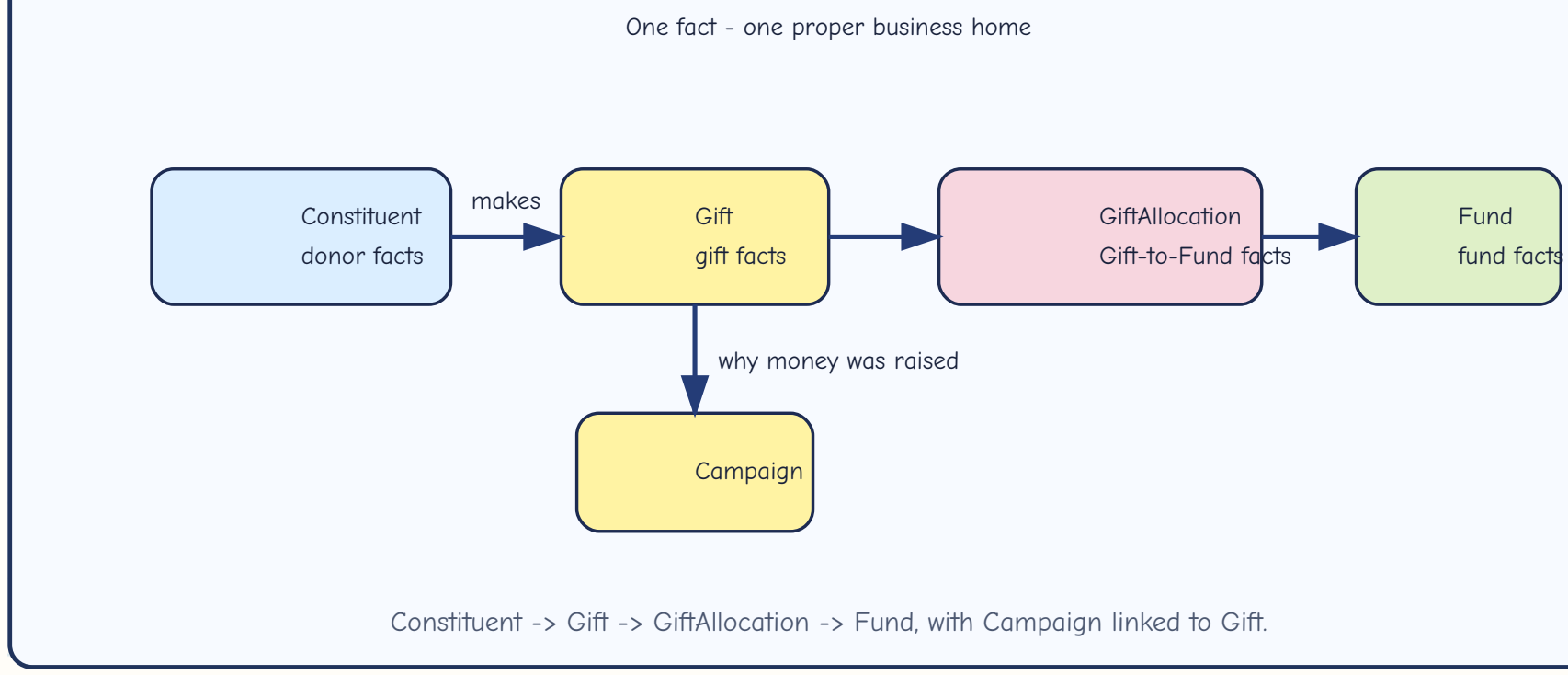
```
Constituent
     |
     | makes
     v
Gift
     |
     +----------> Campaign
     |              WHY money was raised
     |
     v
GiftAllocation
     |
     v
Fund
WHERE the money is used
```

Your model specifically separates Gift and Fund through **GiftAllocation** because one gift can be split across several funds and one fund can receive allocations from many gifts.

What Is Database Normalization?

DIAGRAM

FundraisingDb Fact Ownership



Simple Definition

Database normalization is the process of organizing data into properly related tables so that:

EXAMPLE / FLOW

```
unnecessary duplicate facts are reduced,

data stays consistent,

insert/update/delete operations are safer,

and each business fact has a clear place to live.
```

For example, suppose we created one huge table like this:

EXAMPLE / FLOW

```
GiftDonationDetails
```

GiftId	GiftDate	ConstituentId	ConstituentName	CampaignId	CampaignName	FundId	FundName	AllocationAmount
10000	2026-09-19	1001	John Smith	200	Cancer Campaign	300	Cancer Research	700
10000	2026-09-19	1001	John Smith	200	Cancer Campaign	301	Patient Assistance	300
10001	2026-09-20	1001	John Smith	200	Cancer Campaign	300	Cancer Research	500

This table contains several different business facts mixed together:

EXAMPLE / FLOW

```
Gift facts

Constituent facts

Campaign facts

Fund facts

Gift-to-Fund allocation facts
```

That causes duplication.

For example:

EXAMPLE / FLOW

```
John Smith
```

is repeated for every gift.

EXAMPLE / FLOW

```
Cancer Campaign
```

is repeated for every gift in that campaign.

EXAMPLE / FLOW

```
Cancer Research
```

is repeated for every allocation to that fund.

Normalization asks:

Which fact belongs to which entity?

The normalized answer is:

EXAMPLE / FLOW

```
ConstituentName
→ constituent.Constituent

GiftDate
→ fundraising.Gift

CampaignName
→ fundraising.Campaign

FundName
→ fundraising.Fund

AllocationAmount
→ fundraising.GiftAllocation
```

Why Normalize a Database?

Normalization helps prevent several problems.

The main goals are:

EXAMPLE / FLOW

```
Reduce unnecessary redundancy

Prevent inconsistent data

Prevent insert anomalies

Prevent update anomalies

Prevent delete anomalies

Represent business dependencies correctly

Improve maintainability

Protect data integrity
```

Data Redundancy

Simple Definition

Data redundancy means storing the same business fact unnecessarily in many places.

Suppose we stored this:

EXAMPLE / FLOW

```
GiftId  ConstituentId  ConstituentName
10000   1001           John Smith
10001   1001           John Smith
10002   1001           John Smith
10003   1001           John Smith
```

The fact:

EXAMPLE / FLOW

```
Constituent 1001 is John Smith
```

is repeated four times.

If John eventually has 500 gifts, the same name could appear 500 times.

But the name actually describes:

EXAMPLE / FLOW

```
Constituent
```

not:

EXAMPLE / FLOW

```
Gift
```

Therefore it belongs in:

EXAMPLE / FLOW

```
constituent.Constituent
```

and `Gift` should store only:

EXAMPLE / FLOW

```
ConstituentId
```

as the relationship.

Your actual model follows exactly this idea: `Gift` stores `ConstituentId`, while constituent names and contact information live in `constituent.Constituent`.

Duplicate Data Does Not Always Mean Bad Design

This is important.

Suppose:

EXAMPLE / FLOW

```
GiftId   ConstituentId
10000    1001
10001    1001
10002    1001
```

`ConstituentId = 1001` appears several times.

That is **not bad redundancy**.

It represents a genuine business relationship:

EXAMPLE / FLOW

```
John Smith
made
three different gifts.
```

Therefore:

EXAMPLE / FLOW

```
Foreign-key repetition
```

is normal.

The problem is unnecessarily repeating facts such as:

EXAMPLE / FLOW

```
John's full name

John's email

Campaign description

Fund description
```

inside every gift row.

Business Facts Are the Key to Normalization

When deciding whether something is redundant, ask:

> What business thing does this fact describe?

Example:

EXAMPLE / FLOW

```
PrimaryEmail
```

describes:

EXAMPLE / FLOW

```
Constituent
```

Therefore:

EXAMPLE / FLOW

```
constituent.Constituent.PrimaryEmail
```

is appropriate.

EXAMPLE / FLOW

```
GiftDate
```

describes:

EXAMPLE / FLOW

```
Gift
```

Therefore:

EXAMPLE / FLOW

```
fundraising.Gift.GiftDate
```

is appropriate.

EXAMPLE / FLOW

```
AllocationAmount
```

describes:

EXAMPLE / FLOW

```
the relationship between a Gift and a Fund
```

Therefore it belongs in:

EXAMPLE / FLOW

```
fundraising.GiftAllocation
```

This is one of the most important ideas in normalization:

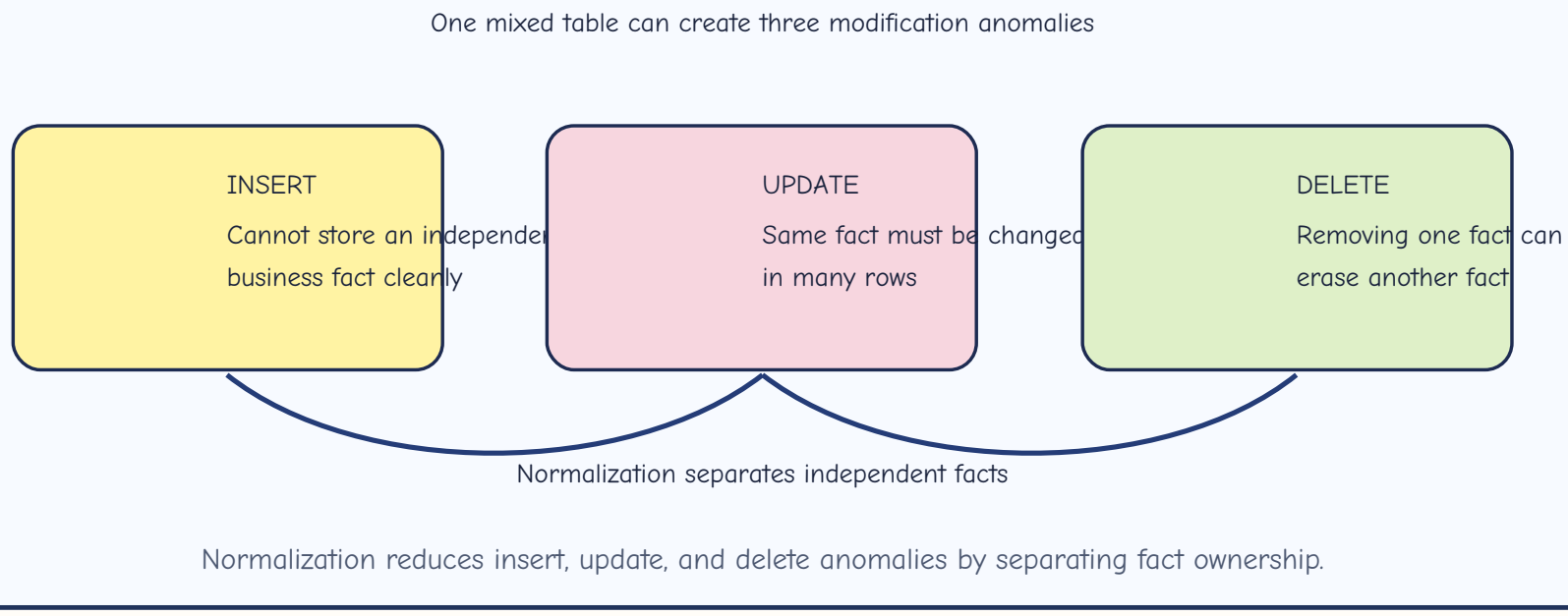
> **Put a fact with the business entity or relationship that actually owns that fact.**

Database Anomalies

Poor normalization creates three major types of anomalies:

DIAGRAM

Modification Anomalies



Normalization reduces insert, update, and delete anomalies by separating fact ownership.

EXAMPLE / FLOW

```
Insert anomaly

Update anomaly

Delete anomaly
```

These are called:

EXAMPLE / FLOW

```
Modification anomalies
```

Update Anomaly

Simple Definition

An **update anomaly** occurs when one business fact is stored in many rows and all copies must be updated.

Suppose our bad table contains:

EXAMPLE / FLOW

```
GiftId   ConstituentId   ConstituentName
10000    1001            John Smith
10001    1001            John Smith
10002    1001            John Smith
```

John changes his display name to:

EXAMPLE / FLOW

```
John A. Smith
```

You now have to update every gift row.

Suppose two rows are updated but one is forgotten:

EXAMPLE / FLOW

```
10000   1001   John A. Smith

10001   1001   John A. Smith

10002   1001   John Smith
```

Now SQL Server contains two different answers to:

> What is constituent 1001's name?

That is an update anomaly.

How Normalization Fixes the Update Anomaly

Store the name once:

EXAMPLE / FLOW

```
constituent.Constituent

ConstituentId = 1001
DisplayName   = John A. Smith
```

Gift contains only:

EXAMPLE / FLOW

```
ConstituentId = 1001
```

Now only one row needs to change.

Insert Anomaly

Simple Definition

An **insert anomaly** occurs when you cannot store one business fact without first creating an unrelated fact.

Suppose Campaign information existed only inside a combined Gift table:

EXAMPLE / FLOW

```
GiftId
CampaignId
CampaignName
GiftAmount
```

Now ABC Children's Foundation wants to create:

EXAMPLE / FLOW

```
2027 Annual Giving Campaign
```

before receiving any gifts.

If Campaign only exists as part of a Gift row, you cannot properly store the new campaign until somebody donates.

That is wrong.

A Campaign is an independent business entity.

Therefore your real design has:

EXAMPLE / FLOW

```
fundraising.Campaign
```

as its own table.

Now you can create a campaign before any gifts exist.

Delete Anomaly

Simple Definition

A **delete anomaly** occurs when deleting one business fact accidentally deletes another fact that should continue to exist.

Suppose Fund information were stored only in allocation rows:

EXAMPLE / FLOW

```
GiftId
FundId
FundName
AllocationAmount
```

Imagine `Cancer Research Fund` currently has only one allocation.

If that GiftAllocation is deleted, the only row containing:

EXAMPLE / FLOW

```
Cancer Research Fund
```

could disappear.

But the Fund should still exist independently.

The correct design stores the Fund separately in:

EXAMPLE / FLOW

```
fundraising.Fund
```

while `GiftAllocation` references its `FundId`.

How Normalization Prevents Anomalies

Normalization separates independent facts.

Instead of:

EXAMPLE / FLOW

```
One giant donation table
```

you have:

EXAMPLE / FLOW

```
Constituent
     |
Gift
```

```
     |
GiftAllocation
     |
Fund
```

with Campaign connected to Gift.

Each table can now:

EXAMPLE / FLOW

```
be inserted independently,

be updated in one authoritative place,

and be deleted according to its own business lifecycle.
```

Keys You Need Before Learning Normal Forms

Normalization depends heavily on understanding keys.

Important key concepts are:

EXAMPLE / FLOW

```
Superkey

Candidate Key

Primary Key

Composite Key

Prime Attribute

Non-Prime Attribute
```

Superkey

Simple Definition

A **superkey** is any column or combination of columns that can uniquely identify a row.

Suppose:

EXAMPLE / FLOW

```
fundraising.Gift
```

has:

EXAMPLE / FLOW

```
GiftId
ReferenceNumber
```

and both are guaranteed unique.

Possible superkeys could include:

EXAMPLE / FLOW

```
GiftId

ReferenceNumber
```

```
GiftId + GiftDate

GiftId + ConstituentId
```

Some contain unnecessary columns.

Candidate Key

Simple Definition

A **candidate key** is a minimal superkey.

It uniquely identifies a row without unnecessary columns.

For example, if:

EXAMPLE / FLOW

```
GiftId
```

is unique by itself, then:

EXAMPLE / FLOW

```
GiftId + GiftDate
```

is not a candidate key because `GiftDate` is unnecessary.

Candidate keys are possible choices for the primary key.

Primary Key

One candidate key is selected as the:

EXAMPLE / FLOW

```
PRIMARY KEY
```

In your model:

EXAMPLE / FLOW

```
fundraising.Gift.GiftId
```

is the primary key.

Similarly:

EXAMPLE / FLOW

```
Constituent.ConstituentId

Campaign.CampaignId

Fund.FundId

GiftAllocation.GiftAllocationId
```

are the main primary keys of those entities.

Composite Key

Simple Definition

A **composite key** uses multiple columns together to identify a row.

For normalization exercises, suppose the business rule for allocations is:

> A gift can allocate money to a particular fund only once.

Then the business combination:

EXAMPLE / FLOW

```
GiftId + FundId
```

could uniquely identify one allocation relationship.

Your actual model uses:

EXAMPLE / FLOW

```
GiftAllocationId
```

as the primary key.

But the pair:

EXAMPLE / FLOW

```
GiftId + FundId
```

is still useful for learning dependencies and might also be protected by a `UNIQUE` constraint if your business rule requires one allocation row per gift/fund pair.

Prime Attribute

A **prime attribute** is a column that participates in at least one candidate key.

Suppose our learning table has candidate key:

EXAMPLE / FLOW

```
GiftId + FundId
```

Then:

EXAMPLE / FLOW

```
GiftId
FundId
```

are prime attributes.

Non-Prime Attribute

A column not participating in any candidate key is called:

EXAMPLE / FLOW

```
non-prime attribute
```

Example:

EXAMPLE / FLOW

```
AllocationAmount
```

would normally be a non-prime attribute.

Functional Dependency

This is the most important theoretical idea behind normalization.

The earlier notes used expressions like:

EXAMPLE / FLOW

```
A → B
```

without enough explanation.

We will not start there.

First understand the business meaning.

Functional Dependency — Simple Definition

A **functional dependency** means:

> If one value is known, it determines exactly one value of another fact according to the business rules.

Consider:

EXAMPLE / FLOW

```
ConstituentId = 1001
```

That ID represents one constituent.

Therefore it determines that constituent's:

EXAMPLE / FLOW

```
DisplayName
PrimaryEmail
PrimaryPhone
ConstituentType
```

Plain English:

> Once SQL Server knows which constituent row we are talking about, it can determine that constituent's name and email.

Only after understanding that should we write the shorter mathematical notation:

EXAMPLE / FLOW

```
ConstituentId → DisplayName
```

Read it as:

> ConstituentId determines DisplayName.

Another example:

EXAMPLE / FLOW

```
FundId → FundName
```

means:

Once we know which FundId we have, that determines the name of that Fund.

Dependency Is Based on Business Rules

Do not look at today's data and guess dependencies.

Suppose only one constituent currently has the name:

EXAMPLE / FLOW

```
John Smith
```

That does **not** mean:

EXAMPLE / FLOW

```
DisplayName determines ConstituentId.
```

Tomorrow another John Smith may be added.

Therefore the dependency:

EXAMPLE / FLOW

```
DisplayName → ConstituentId
```

is not a valid business rule.

But:

EXAMPLE / FLOW

```
ConstituentId → DisplayName
```

is valid because the identifier represents one constituent.

Determinant

The thing doing the determining is called the:

EXAMPLE / FLOW

```
determinant
```

Example:

EXAMPLE / FLOW

```
ConstituentId → DisplayName
```

The determinant is:

EXAMPLE / FLOW

```
ConstituentId
```

Important Dependencies in FundraisingDb

Using your domain:

EXAMPLE / FLOW

```
TenantId determines TenantName.
```

```
AppUserId determines DisplayName and Email.

ConstituentId determines constituent details.

CampaignId determines CampaignName and GoalAmount.

FundId determines FundName and IsRestricted.

GiftId determines GiftDate, TotalAmount,
ConstituentId and CampaignId.

GiftAllocationId determines GiftId,
FundId and AllocationAmount.
```

Compact notation:

EXAMPLE / FLOW

```
TenantId       → TenantName

AppUserId      → AppUser details

ConstituentId  → Constituent details

CampaignId     → Campaign details

FundId         → Fund details

GiftId         → Gift details

GiftAllocationId
               → GiftAllocation details
```

Full Functional Dependency

Simple Definition

A **full functional dependency** occurs when a fact requires the entire composite key to determine it.

Consider a learning version of GiftAllocation where the business key is:

EXAMPLE / FLOW

```
GiftId + FundId
```

Suppose:

EXAMPLE / FLOW

```
Gift 10000
```

is split:

EXAMPLE / FLOW

```
Fund 300 → $700

Fund 301 → $300
```

If I tell you only:

EXAMPLE / FLOW

```
GiftId = 10000
```

you cannot determine:

EXAMPLE / FLOW

```
AllocationAmount
```

because that gift has more than one allocation.

If I tell you only:

EXAMPLE / FLOW

```
FundId = 300
```

you also cannot determine AllocationAmount because Fund 300 receives allocations from many gifts.

But if I tell you:

EXAMPLE / FLOW

```
GiftId = 10000
AND
FundId = 300
```

then you can identify:

EXAMPLE / FLOW

```
AllocationAmount = $700
```

Therefore AllocationAmount depends on the **whole combination**.

Plain English:

> The allocation amount belongs to a specific Gift-to-Fund relationship.

Compact notation:

EXAMPLE / FLOW

```
(GiftId, FundId)
determines
AllocationAmount
```

or:

EXAMPLE / FLOW

```
(GiftId, FundId) → AllocationAmount
```

That is a full functional dependency.

Partial Functional Dependency

Simple Definition

A **partial dependency** occurs when a table has a composite key, but a non-key fact depends on only one part of that key.

Suppose we created this bad table:

EXAMPLE / FLOW

```
GiftFundDetails
```

GiftId	FundId	GiftDate	TotalAmount	FundName	AllocationAmount
10000	300	2026-09-19	1000	Cancer Research	700
10000	301	2026-09-19	1000	Patient Assistance	300

Assume the business key is:

EXAMPLE / FLOW

```
GiftId + FundId
```

Now examine each fact.

GiftDate

Does GiftDate require both GiftId and FundId?

No.

Knowing:

EXAMPLE / FLOW

```
GiftId = 10000
```

is enough to know:

EXAMPLE / FLOW

```
GiftDate = 2026-09-19
```

The Fund does not matter.

Therefore:

EXAMPLE / FLOW

```
GiftDate depends only on GiftId.
```

That is a partial dependency.

TotalAmount

Again:

EXAMPLE / FLOW

```
GiftId = 10000
```

already tells us:

EXAMPLE / FLOW

```
TotalAmount = 1000
```

We do not need FundId.

Therefore:

EXAMPLE / FLOW

```
TotalAmount
```

also partially depends on the composite key.

FundName

If:

EXAMPLE / FLOW

```
FundId = 300
```

then we know:

EXAMPLE / FLOW

```
FundName = Cancer Research Fund
```

We do not need GiftId.

So FundName depends only on:

EXAMPLE / FLOW

```
FundId
```

Another partial dependency.

AllocationAmount

This one is different.

We need:

EXAMPLE / FLOW

```
GiftId
+
FundId
```

together.

Therefore AllocationAmount is fully dependent on the complete business key.

This exact distinction is what 2NF is designed to fix.

Transitive Functional Dependency

Simple Definition

A **transitive dependency** occurs when:

EXAMPLE / FLOW

```
the key determines one non-key value,
and that value determines another fact.
```

Use `Gift` as the example.

Suppose we incorrectly stored:

EXAMPLE / FLOW

```
GiftId
ConstituentId
ConstituentDisplayName
```

inside Gift.

Gift 10000 has:

EXAMPLE / FLOW

```
GiftId = 10000

ConstituentId = 1001
```

The GiftId tells us which ConstituentId belongs to the Gift.

Then:

EXAMPLE / FLOW

```
ConstituentId = 1001
```

tells us:

EXAMPLE / FLOW

```
DisplayName = John Smith
```

So the dependency is:

EXAMPLE / FLOW

```
GiftId
   |
   v
ConstituentId
   |
   v
DisplayName
```

In plain English:

> GiftId identifies the Gift. The Gift identifies its ConstituentId. ConstituentId identifies the Constituent's name.

Therefore GiftId only determines the name **indirectly**.

Compact notation:

EXAMPLE / FLOW

```
GiftId → ConstituentId

ConstituentId → DisplayName
```

Therefore:

EXAMPLE / FLOW

```
GiftId → DisplayName
```

transitively.

That is the kind of problem handled by 3NF.

Multivalued Dependency

A **multivalued dependency** appears when one entity has two or more independent sets of multiple values.

Your `Constituent` domain provides a very good example.

A constituent can have:

EXAMPLE / FLOW

```
multiple addresses
```

and also:

EXAMPLE / FLOW

```
multiple interactions
```

Your schema explicitly supports both relationships.

Suppose John Smith has:

EXAMPLE / FLOW

```
2 addresses
```

and:

EXAMPLE / FLOW

```
3 interactions
```

These are independent facts.

John's address does not determine which interactions occurred.

An interaction does not determine which address John owns.

If we stored both multi-valued facts in one table, SQL Server might have to store:

EXAMPLE / FLOW

```
2 × 3 = 6 rows
```

just to represent every combination.

That is the type of problem 4NF addresses.

Join Dependency

A **join dependency** is a more advanced concept.

It asks:

> Can one table be split into several smaller relationship tables and then joined back together without losing information or inventing false combinations?

This becomes important for 5NF.

We will study it later using:

EXAMPLE / FLOW

```
Constituent

Campaign

Fund
```

from the same fundraising domain.

Normalization Starting Point — UNF

UNF means:

EXAMPLE / FLOW

```
Unnormalized Form
```

This is data before First Normal Form.

Common problems include:

EXAMPLE / FLOW

```
repeating groups,

lists inside one column,

multiple values packed into one field,
```

```
and nested repeating business data.
```

UNF Example Using FundraisingDb

Imagine an import file containing:

GiftId	Constituent	Campaign	FundAllocations
10000	John Smith	Cancer Campaign	Cancer Research:700, *PatientAssistance*:300

The column:

EXAMPLE / FLOW

```
FundAllocations
```

contains two separate business facts:

EXAMPLE / FLOW

```
Fund 300 → $700

Fund 301 → $300
```

inside one field.

That is difficult to:

EXAMPLE / FLOW

```
join,

index,

validate,

reference with foreign keys,

sum accurately,

or update independently.
```

Another UNF Example — Repeating Columns

Bad design:

EXAMPLE / FLOW

```
GiftId

Fund1Id
Fund1Amount

Fund2Id
Fund2Amount

Fund3Id
Fund3Amount
```

What happens when a Gift supports:

EXAMPLE / FLOW

```
4 funds?
```

You need more columns.

What about:

EXAMPLE / FLOW

```
20 funds?
```

This structure does not scale.

Atomic vs Non-Atomic Values

Atomic Value

An atomic value represents one value appropriate to that attribute.

Example:

EXAMPLE / FLOW

```
Gift.TotalAmount = 1000.00
```

One amount.

Non-Atomic Repeating Value

Example:

EXAMPLE / FLOW

```
Gift.Funds = '300,301,305'
```

This column contains several independently meaningful Fund IDs.

Each Fund ID should participate in its own relationship row.

Therefore Fund IDs belong in:

EXAMPLE / FLOW

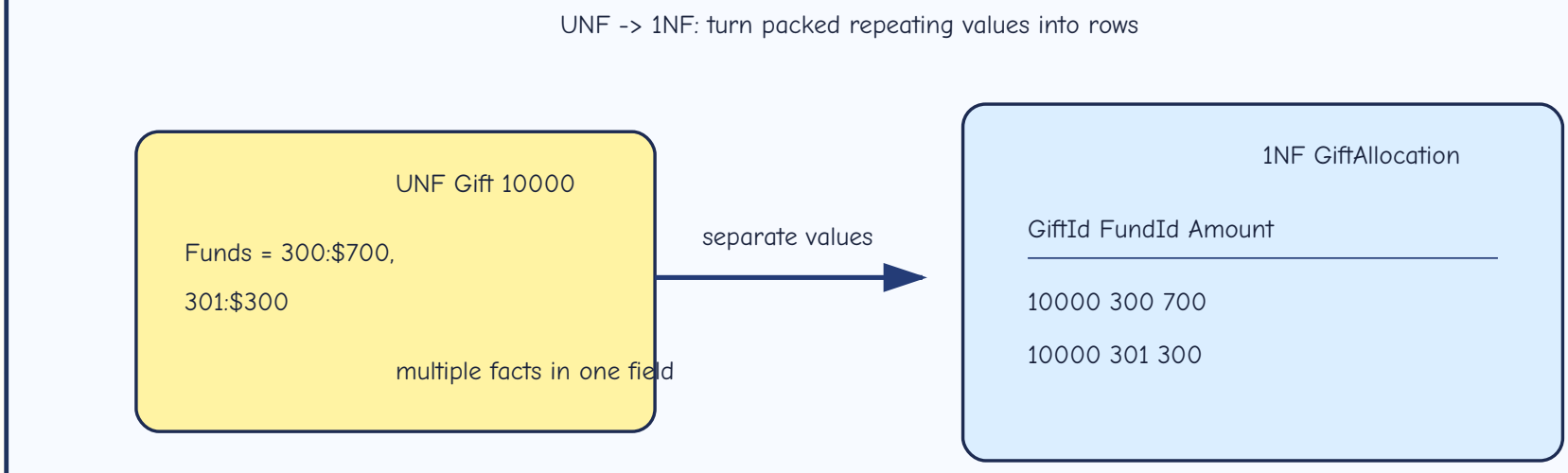
```
GiftAllocation
```

not inside a comma-separated Gift column.

UNF to 1NF — Three Frames

DIAGRAM

UNF to 1NF



Frame 1 — Bad UNF

EXAMPLE / FLOW

```
Gift 10000

Funds =
300:$700,
301:$300
```

Frame 2 — Separate Repeating Values

EXAMPLE / FLOW

```
GiftId    FundId    AllocationAmount
10000     300       700
10000     301       300
```

Frame 3 — Proper Relationship Rows

EXAMPLE / FLOW

```
Gift
  1
  |
  | many
  v
GiftAllocation
  |
  | many
  v
Fund
```

Your real schema uses exactly this Gift-to-Fund allocation model.

First Normal Form — 1NF

Simple Definition

A table is in **First Normal Form** when it has a proper relational structure and repeating values are represented as separate rows instead of repeated columns or packed lists.

Practical rules:

EXAMPLE / FLOW

```
One column represents one attribute.

Do not store several independent IDs in one field.

Do not create Fund1, Fund2, Fund3 repeating columns.

Represent repeated relationships as rows.

Each row should be uniquely identifiable.
```

1NF Example

Instead of:

EXAMPLE / FLOW

```
Gift
-----------------------------------------------
GiftId   FundAllocations
10000    '300:700,301:300'
```

use:

EXAMPLE / FLOW

```
GiftAllocation
-----------------------------------------------
GiftAllocationId   GiftId   FundId   Amount
1                  10000    300      700
2                  10000    301      300
```

Now each allocation is one row.

Why This Is Better

You can now easily ask:

EXAMPLE / FLOW

```
SELECT
    FundId,
    AllocationAmount
FROM fundraising.GiftAllocation
WHERE GiftId = 10000;
```

You can also:

EXAMPLE / FLOW

```
create a foreign key to Fund,

index FundId,
```

```
sum allocations,

update one allocation,

delete one allocation,

and validate each allocation separately.
```

Important 1NF Rule

1NF does **not** mean the database is fully normalized.

Consider:

EXAMPLE / FLOW

```
GiftId
FundId
GiftDate
ConstituentName
FundName
AllocationAmount
```

with one value in every cell.

That may satisfy 1NF.

But it can still contain:

EXAMPLE / FLOW

```
partial dependencies

transitive dependencies

large amounts of redundancy
```

So normalization continues.

If 1NF Is Not Followed

Common problems include:

EXAMPLE / FLOW

```
LIKE '%300%' searches for IDs,

string parsing,

no reliable foreign keys,

difficult indexing,

hard updates,

duplicate values inside lists,

and inconsistent formatting.
```

Easy Memory Rule

EXAMPLE / FLOW

```
1NF
=
No repeating groups
and no packed repeating business values.
```

Second Normal Form — 2NF

Simple Definition

A table is in **Second Normal Form** when:

EXAMPLE / FLOW

```
it is already in 1NF,

and every non-key fact depends on
the complete candidate key,
not just part of a composite key.
```

Easy memory:

EXAMPLE / FLOW

```
2NF
=
1NF
+
No partial dependency.
```

Real Fundraising Example for 2NF

Consider this deliberately bad learning table:

EXAMPLE / FLOW

```
GiftFundDetails
```

GiftId	FundId	GiftDate	TotalAmount	FundName	AllocationAmount
10000	300	2026-09-19	1000	Cancer Research	700
10000	301	2026-09-19	1000	Patient Assistance	300

Business key:

EXAMPLE / FLOW

```
GiftId + FundId
```

because each row represents:

One Gift allocated to one Fund.

Analyze Every Column Carefully

GiftDate

Question:

> Do I need both GiftId and FundId to know the GiftDate?

No.

Gift 10000 has one GiftDate regardless of which Fund receives part of it.

Therefore:

EXAMPLE / FLOW

```
GiftDate belongs to Gift.
```

TotalAmount

Question:

> Do I need both GiftId and FundId to know the total gift amount?

No.

Gift 10000 has total:

EXAMPLE / FLOW

```
$1,000
```

regardless of the Fund.

Therefore:

EXAMPLE / FLOW

```
TotalAmount belongs to Gift.
```

FundName

Question:

> Do I need GiftId to know Fund 300's name?

No.

Fund 300 is:

EXAMPLE / FLOW

```
Cancer Research Fund
```

regardless of which gift allocates money to it.

Therefore:

EXAMPLE / FLOW

```
FundName belongs to Fund.
```

AllocationAmount

Question:

Is AllocationAmount only a Gift fact?

No.

Gift 10000 has multiple allocations.

Question:

Is AllocationAmount only a Fund fact?

No.

Fund 300 receives allocations from many gifts.

We need:

EXAMPLE / FLOW

```
GiftId
+
FundId
```

to identify a particular allocation.

Therefore:

EXAMPLE / FLOW

```
AllocationAmount belongs to GiftAllocation.
```

2NF Problem Diagram

EXAMPLE / FLOW

```
Composite business key

GiftId + FundId
      |
      +-------- AllocationAmount
                needs BOTH
```

But:

EXAMPLE / FLOW

```
GiftId
  |
  +---- GiftDate
  |
  +---- TotalAmount
```

and:

EXAMPLE / FLOW

```
FundId
  |
  +---- FundName
```

These facts depend on only part of the composite key.

That is the 2NF violation.

Convert to 2NF

Split the facts.

fundraising.Gift

EXAMPLE / FLOW

```
GiftId
GiftDate
TotalAmount
ConstituentId
CampaignId
```

fundraising.Fund

EXAMPLE / FLOW

```
FundId
FundName
IsRestricted
```

fundraising.GiftAllocation

EXAMPLE / FLOW

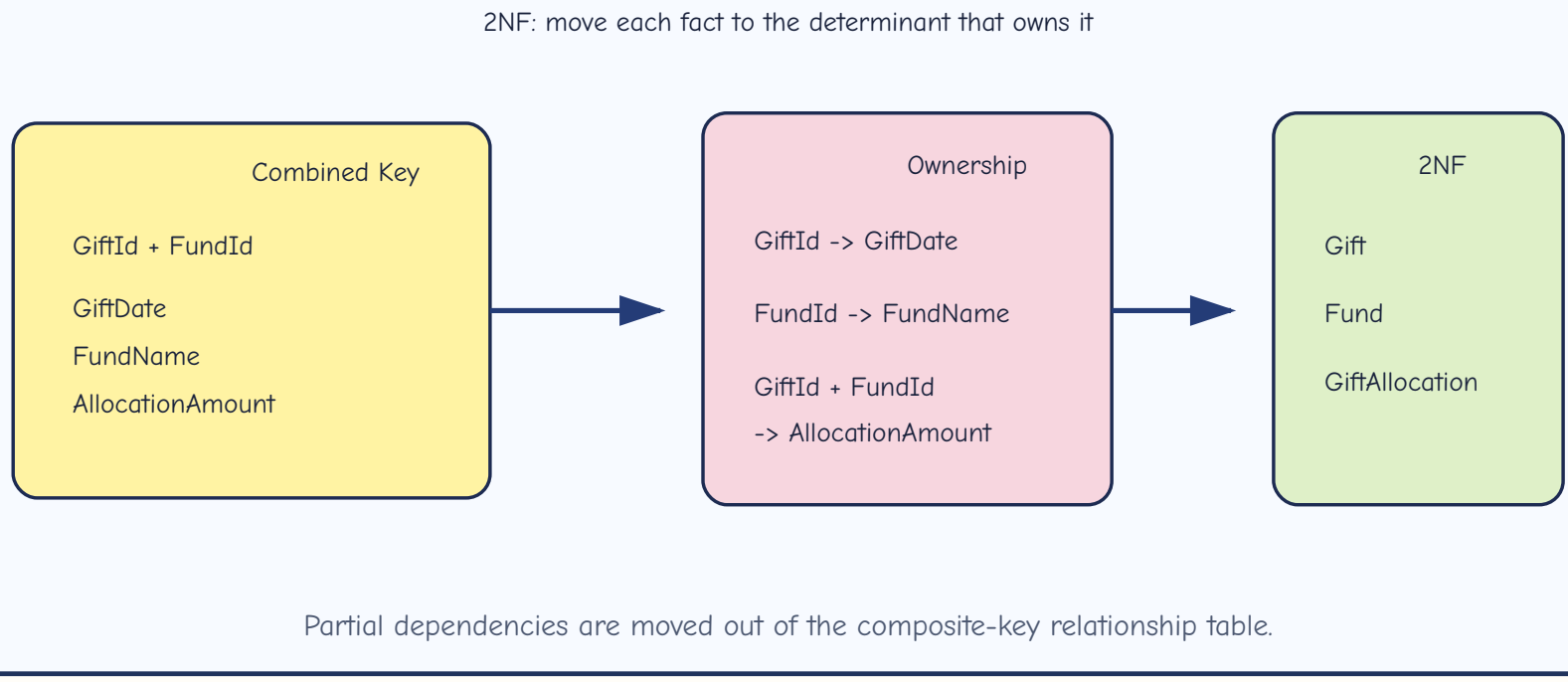
```
GiftAllocationId
GiftId
FundId
AllocationAmount
```

This matches the real responsibility of the tables in your schema.

2NF Three-Frame Diagram

DIAGRAM

2NF Fact Ownership



Frame 1 — Combined Table

EXAMPLE / FLOW

```
GiftId + FundId

GiftDate
TotalAmount
FundName
AllocationAmount
```

Frame 2 — Identify Ownership

EXAMPLE / FLOW

```
GiftId
→ GiftDate
→ TotalAmount

FundId
→ FundName

GiftId + FundId
→ AllocationAmount
```

Frame 3 — Separate Tables

EXAMPLE / FLOW

```
Gift              GiftAllocation          Fund
----              --------------          ----
GiftId            GiftId                  FundId
GiftDate          FundId                  FundName
TotalAmount       AllocationAmount
```

Why 2NF Matters

Without 2NF:

EXAMPLE / FLOW

```
GiftDate repeats for every Fund allocation.

TotalAmount repeats for every Fund allocation.

FundName repeats for every Gift using the Fund.
```

This creates:

EXAMPLE / FLOW

```
redundancy,

update anomalies,

more storage,

and inconsistent values.
```

Important 2NF Rule

2NF problems require a composite candidate key.

If all candidate keys are single-column keys, there cannot be a dependency on "only part" of that single column.

That does **not** mean the table is fully normalized.

It may still violate:

EXAMPLE / FLOW

```
3NF

BCNF
```

Third Normal Form — 3NF

Simple Definition

A table is in **Third Normal Form** when:

EXAMPLE / FLOW

```
it is already in 2NF,

and non-key facts do not depend
on other non-key facts
in a way that represents another entity.
```

Easy memory:

EXAMPLE / FLOW

```
3NF
=
2NF
+
No inappropriate transitive dependency.
```

Real Fundraising Example for 3NF

Suppose we incorrectly designed Gift like this:

EXAMPLE / FLOW

```
GiftId
GiftDate
TotalAmount

ConstituentId
ConstituentDisplayName
ConstituentEmail

CampaignId
CampaignName
CampaignGoalAmount
```

At first this might appear convenient.

But it mixes:

EXAMPLE / FLOW

```
Gift facts

Constituent facts

Campaign facts
```

Constituent Transitive Dependency

Take:

EXAMPLE / FLOW

```
GiftId = 10000
```

The Gift record tells us:

EXAMPLE / FLOW

```
ConstituentId = 1001
```

Then the ConstituentId tells us:

EXAMPLE / FLOW

```
DisplayName = John Smith
```

and:

EXAMPLE / FLOW

```
PrimaryEmail = ...
```

So the logical path is:

EXAMPLE / FLOW

```
GiftId
   |
   v
ConstituentId
   |
   v
Constituent details
```

The constituent details do not truly describe the Gift.

They describe:

EXAMPLE / FLOW

```
Constituent
```

Campaign Transitive Dependency

Similarly:

EXAMPLE / FLOW

```
GiftId
   |
   v
CampaignId
```

```
    |
    v
CampaignName
CampaignGoalAmount
```

The Campaign information describes:

EXAMPLE / FLOW

```
Campaign
```

not:

EXAMPLE / FLOW

```
Gift
```

Convert to 3NF

constituent.Constituent

EXAMPLE / FLOW

```
ConstituentId
DisplayName
PrimaryEmail
PrimaryPhone
...
```

fundraising.Campaign

EXAMPLE / FLOW

```
CampaignId
CampaignCode
CampaignName
GoalAmount
...
```

fundraising.Gift

EXAMPLE / FLOW

```
GiftId
ConstituentId
CampaignId
GiftDate
TotalAmount
...
```

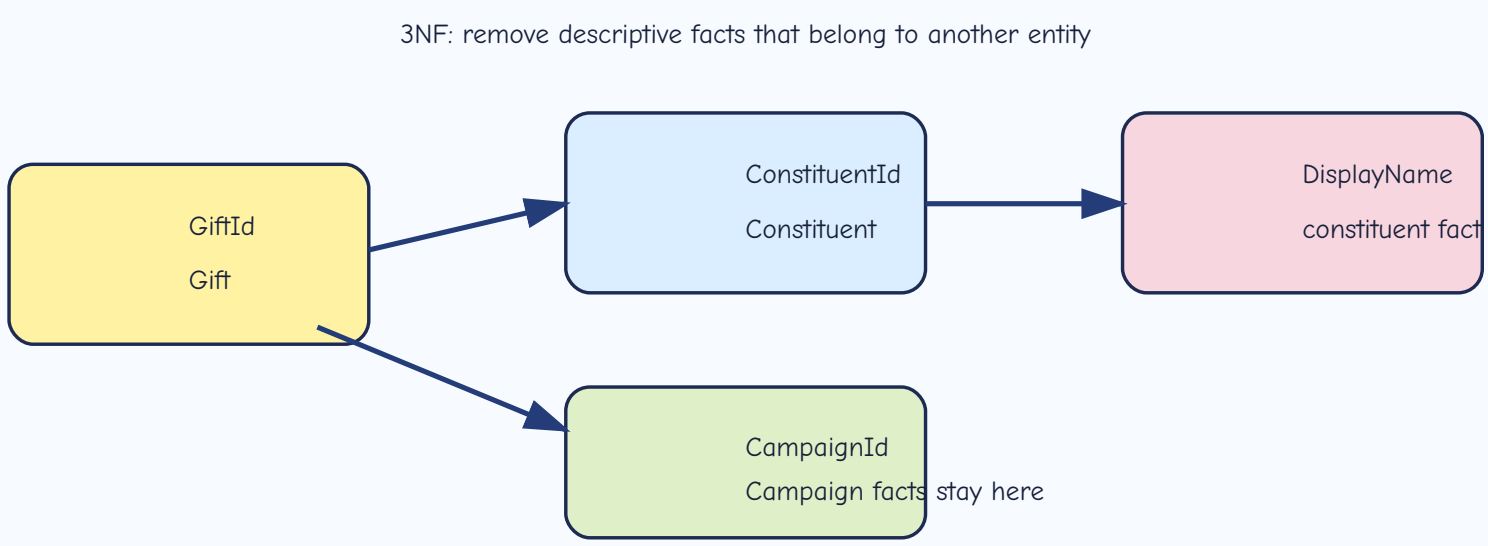
Gift references those entities through IDs.

Your schema already follows this pattern: `Gift` stores `ConstituentId` and `CampaignId`, while descriptive constituent and campaign information remains in the corresponding tables.

3NF Three-Frame Diagram

DIAGRAM

3NF Transitive Dependency



Frame 1 — Bad Gift

EXAMPLE / FLOW

```
Gift
--------------------------------
GiftId
ConstituentId
ConstituentName
CampaignId
CampaignName
TotalAmount
```

Frame 2 — Identify the Dependencies

EXAMPLE / FLOW

```
GiftId
  |
  +---- ConstituentId
  |           |
  |           +---- ConstituentName
  |
  +---- CampaignId
              |
              +---- CampaignName
```

Frame 3 — Correct Tables

EXAMPLE / FLOW

```
Constituent              Gift              Campaign
-----------              ----              --------
ConstituentId <--------- ConstituentId
DisplayName              GiftId
                         CampaignId ------> CampaignId
                                            CampaignName
```

Classic 3NF Memory Rule

A common memory rule is:

> Every non-key fact should depend on **the key, the whole key, and nothing but the key**.

For this example:

EXAMPLE / FLOW

```
GiftDate
```

depends directly on Gift.

EXAMPLE / FLOW

```
ConstituentName
```

does not.

It depends on Constituent.

What Happens If 3NF Is Ignored?

Suppose CampaignName is repeated in 50,000 gifts.

Campaign is renamed.

Now you have to update:

EXAMPLE / FLOW

```
50,000 Gift rows
```

instead of:

EXAMPLE / FLOW

```
1 Campaign row
```

That is exactly the kind of redundancy normalization prevents.

1NF vs 2NF vs 3NF

Normal Form	Main Question
1NF	Are repeating values stored properly as rows?
2NF	Does each non-key fact depend on the complete composite key?
3NF	Does a non-key fact depend on another non-key fact?

Memory:

EXAMPLE / FLOW

```
1NF
→ No repeating groups

2NF
→ No partial dependency
```

```
3NF
→ No inappropriate transitive dependency
```

Boyce-Codd Normal Form — BCNF

BCNF means:

EXAMPLE / FLOW

```
Boyce-Codd Normal Form
```

BCNF is a stricter version of 3NF.

Simple BCNF Definition

For every important dependency:

> The thing determining another fact should itself uniquely identify a row.

Formal language:

EXAMPLE / FLOW

```
Every determinant must be a superkey.
```

Why BCNF Is More Difficult to See

Most ordinary well-designed 3NF application tables are also BCNF.

The difference normally appears when:

EXAMPLE / FLOW

```
multiple candidate keys overlap
```

and a dependency exists where the determinant is not itself a complete candidate key.

Your current FundraisingDb tables do not naturally provide a clear BCNF-only violation.

So for learning BCNF, we will create a **temporary modeling exercise using only your existing business entities**:

EXAMPLE / FLOW

```
Constituent
AppUser
Campaign
```

This is not a new table recommendation for your application.

It is only a same-domain learning example.

BCNF Learning Example

Imagine a temporary design:

EXAMPLE / FLOW

```
FundraiserCampaignCoverage
```

with:

EXAMPLE / FLOW

```
ConstituentId

AppUserId

CampaignId
```

Suppose the business rules are:

EXAMPLE / FLOW

```
A constituent involved in a campaign
has exactly one assigned fundraiser.

Each fundraiser works for exactly one campaign
in this special scenario.
```

Then:

EXAMPLE / FLOW

```
ConstituentId + CampaignId
```

determines:

EXAMPLE / FLOW

```
AppUserId
```

because one fundraiser handles that constituent for that campaign.

But we also have:

EXAMPLE / FLOW

```
AppUserId
```

determining:

EXAMPLE / FLOW

```
CampaignId
```

because each fundraiser works for one campaign under this hypothetical rule.

Why This Can Violate BCNF

Candidate keys can be:

EXAMPLE / FLOW

```
ConstituentId + CampaignId

ConstituentId + AppUserId
```

Now consider:

EXAMPLE / FLOW

```
AppUserId determines CampaignId.
```

But AppUserId alone cannot uniquely identify a coverage row because the same fundraiser may handle many constituents.

Therefore:

EXAMPLE / FLOW

```
AppUserId
```

is not a superkey.

Yet it determines CampaignId.

BCNF says:

> That determinant should be a superkey.

So this design violates BCNF.

BCNF Decomposition

Separate:

EXAMPLE / FLOW

```
AppUserCampaign
---------------
AppUserId
CampaignId
```

and:

EXAMPLE / FLOW

```
ConstituentFundraiser
---------------------
ConstituentId
AppUserId
```

Now the special rule:

EXAMPLE / FLOW

```
One AppUser → One Campaign
```

has its own proper table.

Again:

> This example is only for understanding BCNF. It is not a recommendation to add these tables to your current FundraisingDb.

BCNF Three Frames

Frame 1

EXAMPLE / FLOW

```
ConstituentId
AppUserId
CampaignId
```

Frame 2

EXAMPLE / FLOW

```
AppUserId
determines
CampaignId

but AppUserId does not uniquely
identify the whole row.
```

Frame 3

EXAMPLE / FLOW

```
AppUserCampaign

ConstituentFundraiser
```

3NF vs BCNF

3NF is slightly less strict.

BCNF says:

EXAMPLE / FLOW

```
Every determinant must be a superkey.
```

A special dependency can satisfy formal 3NF because the dependent attribute is prime while still violating BCNF.

In most ordinary application design:

EXAMPLE / FLOW

```
3NF + sensible keys
```

already gives a strong design.

BCNF becomes important when unusual candidate-key dependencies appear.

BCNF Trade-Off

Higher normalization is not automatically better.

A BCNF decomposition may sometimes make a dependency harder to enforce because enforcing the original rule may require checking more than one table.

Therefore a carefully designed 3NF schema can sometimes be preferable when:

EXAMPLE / FLOW

```
dependency preservation
```

is important.

Fourth Normal Form — 4NF

4NF handles:

EXAMPLE / FLOW

```
multivalued dependencies
```

This is different from the partial and transitive dependencies handled earlier.

Real Fundraising Example for 4NF

Your Constituent can have:

EXAMPLE / FLOW
```
multiple addresses
```

through:

EXAMPLE / FLOW
```
ConstituentAddress
```

and can also have:

EXAMPLE / FLOW
```
multiple interactions
```

through:

EXAMPLE / FLOW
```
Interaction
```

Your actual schema models these independently.

Now imagine a bad design:

EXAMPLE / FLOW
```
ConstituentAddressInteraction
```

ConstituentId	AddressId	InteractionId
1001	5001	9001
1001	5001	9002
1001	5002	9001
1001	5002	9002

Suppose John has:

EXAMPLE / FLOW
```
2 addresses
```

and:

EXAMPLE / FLOW
```
2 interactions
```

Because they are independent, the combined table generates:

EXAMPLE / FLOW
```
2 × 2 = 4 rows
```

Why This Is a Problem

The address:

EXAMPLE / FLOW

```
123 Main Street
```

does not determine which Interaction occurred.

Likewise:

EXAMPLE / FLOW

```
Phone call on September 19
```

does not determine which address John owns.

They are two independent collections associated with the same Constituent.

Multivalued Dependencies in Plain English

We could say:

> Constituent 1001 can independently have many addresses.

And:

> Constituent 1001 can independently have many interactions.

Only after understanding that do we use compact notation:

EXAMPLE / FLOW

```
ConstituentId →→ AddressId

ConstituentId →→ InteractionId
```

The double arrow means:

EXAMPLE / FLOW

```
multiple independent values
```

Fourth Normal Form Definition

A relation is in 4NF when:

EXAMPLE / FLOW

```
it satisfies BCNF,

and it does not mix independent
multivalued relationships together
unless the determinant is a superkey.
```

Practical memory:

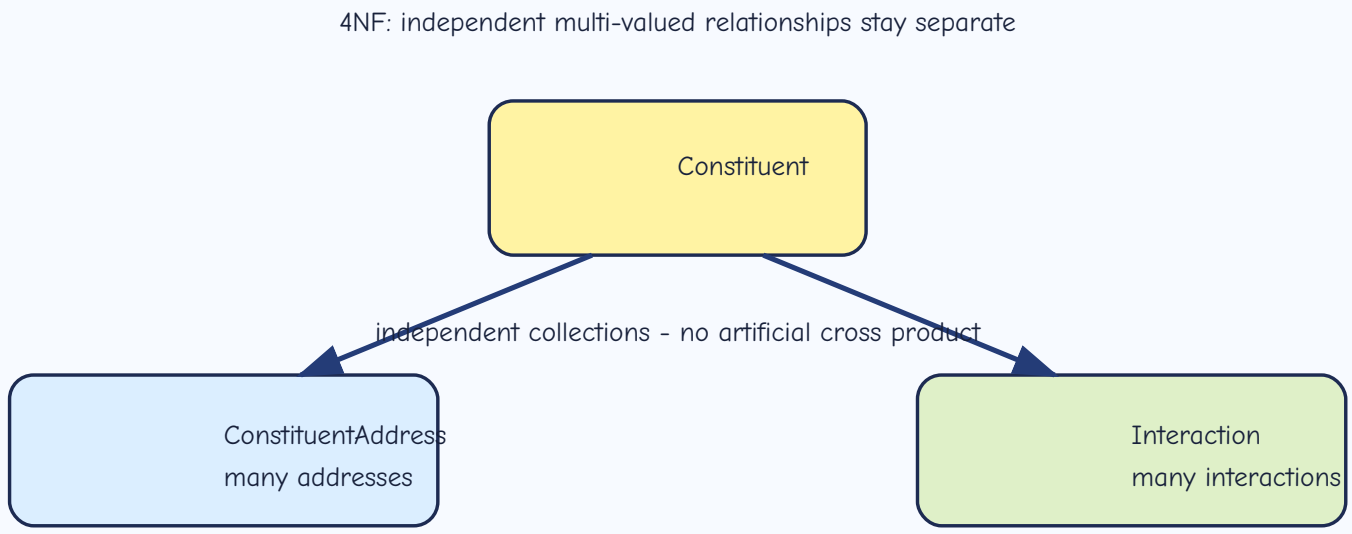
> **Do not combine two independent one-to-many relationships in one table.**

Correct 4NF Design

Your existing schema already does this correctly.

DIAGRAM

4NF Independent Relationships



Addresses and interactions are separate child relationships of Constituent.

Addresses:

EXAMPLE / FLOW

```
Constituent
    |
    v
ConstituentAddress
    |
    v
Address
```

Interactions:

EXAMPLE / FLOW

```
Constituent
    |
    v
Interaction
```

They are independent tables.

4NF Three Frames

Frame 1 — Bad Combined Table

EXAMPLE / FLOW

```
ConstituentId
AddressId
InteractionId
```

Frame 2 — Recognize Independence

EXAMPLE / FLOW

```
Constituent
   |
   +---- many Addresses

   +---- many Interactions
```

The two child collections do not determine each other.

Frame 3 — Separate Relationships

EXAMPLE / FLOW

```
ConstituentAddress

Interaction
```

Both reference Constituent independently.

Fifth Normal Form — 5NF

5NF deals with:

EXAMPLE / FLOW

```
join dependencies
```

and is considerably more specialized.

Most normal OLTP application developers rarely need to explicitly redesign tables because of 5NF.

But understanding the idea is useful.

Simple 5NF Idea

Suppose we have three business entities:

EXAMPLE / FLOW

```
Constituent

Campaign

Fund
```

Imagine a hypothetical relation:

EXAMPLE / FLOW

```
ConstituentCampaignFundEligibility
```

meaning:

> This constituent can participate in this campaign and direct gifts to this fund.

A row might be:

EXAMPLE / FLOW

```
John Smith
Cancer Campaign
Cancer Research Fund
```

The question becomes:

> Is this three-way business fact completely determined by several smaller two-way relationships?

For example:

EXAMPLE / FLOW

```
ConstituentCampaign

ConstituentFund

CampaignFund
```

If the business rules guarantee that every compatible combination of those pairs represents a valid triple, then the ternary table may be decomposable.

But if not, decomposition creates false business facts.

Why This Is Dangerous

Suppose we know:

EXAMPLE / FLOW

```
John participates in Cancer Campaign.
```

and:

EXAMPLE / FLOW

```
John can donate to Patient Assistance Fund.
```

and:

EXAMPLE / FLOW

```
Cancer Campaign can involve Cancer Research Fund.
```

That does **not automatically mean**:

EXAMPLE / FLOW

```
John + Cancer Campaign + Patient Assistance Fund
```

is a valid combination.

You must understand the business rule first.

Your Existing Schema Is a Good Warning Example

Your schema intentionally does **not** directly say:

EXAMPLE / FLOW

```
Campaign → Fund
```

Instead:

EXAMPLE / FLOW

```
Campaign
    |
    v
Gift
    |
    v
GiftAllocation
    |
```

```
    v
Fund
```

The file specifically notes that Fund does not need a direct Campaign relationship; the association occurs through Gift and GiftAllocation for flexibility.

That prevents us from inventing a business rule that does not actually exist.

Fifth Normal Form Definition

A table is in 5NF when:

> Every non-trivial join dependency is implied by candidate keys, and the relation cannot be decomposed further without losing the exact meaning of valid combinations.

Easy memory:

EXAMPLE / FLOW

```
5NF
=
No unnecessary complex join dependency.
```

5NF Three Frames

Frame 1

EXAMPLE / FLOW

```
Constituent
Campaign
Fund
```

combined in one relationship.

Frame 2

Ask:

EXAMPLE / FLOW

```
Can the three-way fact
be reconstructed exactly
from smaller pairwise facts?
```

Frame 3

Only decompose when:

EXAMPLE / FLOW

```
the business rules guarantee
that joining the pieces
produces exactly the valid combinations.
```

Otherwise:

EXAMPLE / FLOW

```
do not decompose.
```

Normal Form Progression

EXAMPLE / FLOW

```
UNF
 |
 | Remove repeating groups
 v
1NF
 |
 | Remove partial dependencies
 v
2NF
 |
 | Remove transitive dependencies
 v
3NF
 |
 | Every determinant should be a superkey
 v
BCNF
 |
 | Separate independent multivalued facts
 v
4NF
 |
```

EXAMPLE / FLOW (CONT. 2)

```
 | Remove inappropriate join dependencies
 v
5NF
```

Normal Forms Summary

Normal Form	Main Problem
UNF	Repeating / packed values
1NF	Repeating groups
2NF	Partial dependency
3NF	Transitive dependency
BCNF	Non-key determinant
4NF	Independent multivalued dependencies
5NF	Complex join dependencies

Lossless Decomposition

Normalization normally splits one table into several tables.

That split must be safe.

Simple Definition

A decomposition is **lossless** when:

> Joining the normalized tables can reconstruct exactly the original valid business information.

No legitimate facts disappear.

No false facts are invented.

Lossless Example Using Constituent and Gift

Suppose the bad table was:

EXAMPLE / FLOW

```
GiftId
GiftDate
ConstituentId
ConstituentName
```

We separate it into:

EXAMPLE / FLOW

```
Gift
----------------
GiftId
GiftDate
ConstituentId
```

and:

EXAMPLE / FLOW

```
Constituent
----------------
ConstituentId
DisplayName
```

Join:

EXAMPLE / FLOW

```
SELECT
    g.GiftId,
    g.GiftDate,
    c.ConstituentId,
    c.DisplayName
FROM fundraising.Gift AS g
INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = g.ConstituentId;
```

Because Gift contains the valid Constituent foreign key, we can correctly reconstruct the original information.

This is:

EXAMPLE / FLOW

```
lossless decomposition
```

Lossy Decomposition

Simple Definition

A decomposition is **lossy** when joining the pieces:

EXAMPLE / FLOW

```
loses valid information
```

or:

EXAMPLE / FLOW

```
creates false combinations.
```

Lossy Example Using Gift, Campaign, and Fund

Suppose original valid data is:

EXAMPLE / FLOW

```
Gift 10000
Campaign 200
Fund 300

Gift 10000
Campaign 200
Fund 301

Gift 10001
Campaign 200
Fund 300
```

Now imagine we incorrectly throw away the Gift-to-Fund relationship and keep only:

EXAMPLE / FLOW

```
GiftCampaign
```

and:

EXAMPLE / FLOW

```
CampaignFund
```

Campaign 200 may have many gifts and those gifts may allocate to different funds.

Joining only on Campaign could create:

EXAMPLE / FLOW

```
Gift 10001
Fund 301
```

even if Gift 10001 was never allocated to Fund 301.

That generated row is called:

EXAMPLE / FLOW

```
spurious row
```

or:

EXAMPLE / FLOW

```
spurious tuple.
```

This is exactly why your actual schema keeps:

EXAMPLE / FLOW

```
GiftAllocation.GiftId
+
GiftAllocation.FundId
```

together.

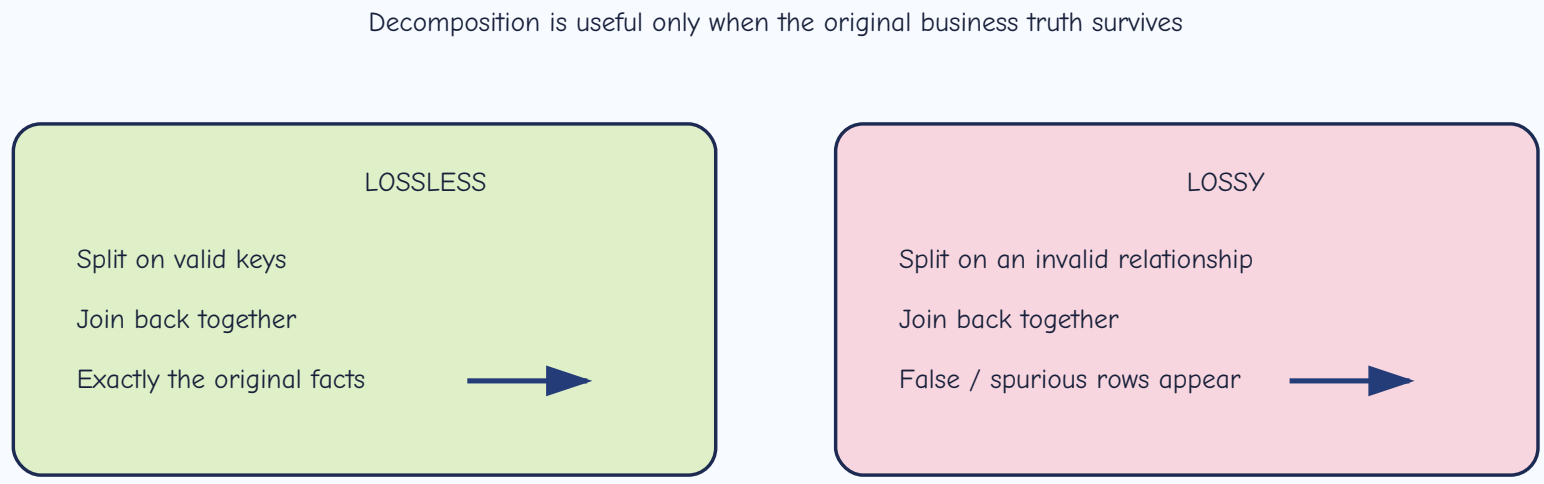
Lossless vs Lossy

EXAMPLE / FLOW

```
Lossless
→ Rejoin and recover exactly the valid facts.
```

DIAGRAM

Lossless vs Lossy Decomposition



EXAMPLE / FLOW

```
Lossy
→ Rejoin and lose information or invent false facts.
```

Normalization decomposition should be:

EXAMPLE / FLOW

```
lossless.
```

Dependency Preservation

Another important goal is:

EXAMPLE / FLOW

```
dependency preservation
```

Simple Definition

A decomposition is dependency-preserving when important business dependencies can still be enforced inside the resulting tables without needing a complex join merely to check the rule.

Fundraising Example

Originally we know:

EXAMPLE / FLOW

```
ConstituentId determines ConstituentName.
```

After normalization, both values are stored in:

EXAMPLE / FLOW

```
constituent.Constituent
```

So the rule is still naturally represented in one table.

Similarly:

EXAMPLE / FLOW

```
FundId determines FundName.
```

Both are in:

EXAMPLE / FLOW

```
fundraising.Fund
```

Similarly:

EXAMPLE / FLOW

```
GiftId determines GiftDate and TotalAmount.
```

Those facts are stored in:

EXAMPLE / FLOW

```
fundraising.Gift
```

The dependencies are preserved naturally.

Lossless vs Dependency Preservation

They answer different questions.

Lossless

Ask:

> Can I reconstruct the original correct information?

Dependency Preservation

Ask:

> Can I still enforce the original business rules conveniently after decomposition?

Ideal Decomposition

A good decomposition aims for:

EXAMPLE / FLOW

```
Reduced redundancy
+
Lossless decomposition
+
Dependency preservation
```

where practical.

Normalization and Referential Integrity

Normalization creates related tables.

Foreign keys protect those relationships.

For example:

EXAMPLE / FLOW

```
Gift.ConstituentId
```

should reference:

EXAMPLE / FLOW

```
Constituent.ConstituentId
```

This guarantees:

A Gift cannot reference a nonexistent Constituent.

Similarly:

EXAMPLE / FLOW

```
GiftAllocation.GiftId
```

references:

EXAMPLE / FLOW

```
Gift.GiftId
```

and:

EXAMPLE / FLOW

```
GiftAllocation.FundId
```

references:

EXAMPLE / FLOW

```
Fund.FundId
```

Your model uses these relationships explicitly.

Normalization Does Not Replace Constraints

A normalized schema still needs:

EXAMPLE / FLOW

```
PRIMARY KEY

FOREIGN KEY

UNIQUE

NOT NULL

CHECK
```

Normalization decides:

EXAMPLE / FLOW

```
where the data should live.
```

Constraints enforce:

EXAMPLE / FLOW

```
what data is allowed.
```

Practical SQL Server Example

A simplified normalized fundraising structure:

EXAMPLE / FLOW

```
CREATE TABLE constituent.Constituent
(
    ConstituentId BIGINT
        IDENTITY(1,1)
        PRIMARY KEY,

    TenantId BIGINT
        NOT NULL,

    DisplayName VARCHAR(200)
        NOT NULL,

    PrimaryEmail VARCHAR(255)
        NULL
);
```

EXAMPLE / FLOW

```
CREATE TABLE fundraising.Campaign
(
    CampaignId BIGINT
        IDENTITY(1,1)
        PRIMARY KEY,

    TenantId BIGINT
        NOT NULL,

    CampaignCode VARCHAR(50)
        NOT NULL,

    CampaignName VARCHAR(200)
```

```
        NOT NULL,

    GoalAmount DECIMAL(18,2)
        NULL
);
```

EXAMPLE / FLOW

```
CREATE TABLE fundraising.Fund
(
    FundId BIGINT
        IDENTITY(1,1)
        PRIMARY KEY,

    TenantId BIGINT
        NOT NULL,

    FundCode VARCHAR(50)
        NOT NULL,

    FundName VARCHAR(200)
        NOT NULL,

    IsRestricted BIT
        NOT NULL
);
```

EXAMPLE / FLOW

```
CREATE TABLE fundraising.Gift
(
    GiftId BIGINT
        IDENTITY(1,1)
        PRIMARY KEY,

    TenantId BIGINT
        NOT NULL,

    ConstituentId BIGINT
        NOT NULL,

    CampaignId BIGINT
        NULL,

    GiftDate DATE
        NOT NULL,

    TotalAmount DECIMAL(18,2)
        NOT NULL,
```

EXAMPLE / FLOW (CONT. 2)

```
    CONSTRAINT FK_Gift_Constituent
        FOREIGN KEY (ConstituentId)
        REFERENCES constituent.Constituent(ConstituentId),

    CONSTRAINT FK_Gift_Campaign
        FOREIGN KEY (CampaignId)
        REFERENCES fundraising.Campaign(CampaignId)
);
```

EXAMPLE / FLOW

```
CREATE TABLE fundraising.GiftAllocation
(
    GiftAllocationId BIGINT
        IDENTITY(1,1)
        PRIMARY KEY,

    TenantId BIGINT
        NOT NULL,

    GiftId BIGINT
        NOT NULL,

    FundId BIGINT
        NOT NULL,

    AllocationAmount DECIMAL(18,2)
        NOT NULL,

    CONSTRAINT FK_GiftAllocation_Gift
        FOREIGN KEY (GiftId)
        REFERENCES fundraising.Gift(GiftId),
```

EXAMPLE / FLOW (CONT. 2)

```
    CONSTRAINT FK_GiftAllocation_Fund
        FOREIGN KEY (FundId)
        REFERENCES fundraising.Fund(FundId),

    CONSTRAINT CK_GiftAllocation_Amount
        CHECK (AllocationAmount > 0)
);
```

This structure keeps:

EXAMPLE / FLOW

```
Constituent facts
with Constituent

Campaign facts
with Campaign

Fund facts
with Fund

Gift facts
with Gift

Gift-to-Fund relationship facts
with GiftAllocation
```

Normalization of Constituent Addresses

Your address model is another good normalization example.

A Constituent may have:

EXAMPLE / FLOW

```
multiple addresses
```

and the relationship itself has information such as:

EXAMPLE / FLOW

```
AddressType

IsPrimary

ValidFrom

ValidTo
```

Therefore these facts do not belong directly in:

EXAMPLE / FLOW

```
Constituent
```

or solely in:

EXAMPLE / FLOW

```
Address.
```

They describe:

EXAMPLE / FLOW

```
the relationship between
Constituent and Address.
```

So they live in:

EXAMPLE / FLOW

```
ConstituentAddress.
```

Your actual model uses this exact junction-table pattern.

Why AddressType Belongs in ConstituentAddress

Suppose address 5001 is:

EXAMPLE / FLOW

```
123 Main Street
```

For John Smith it might be:

EXAMPLE / FLOW

```
Home
```

If the same physical Address row were shared with another relationship, the relationship meaning could differ.

Therefore:

EXAMPLE / FLOW

```
AddressType
```

describes:

EXAMPLE / FLOW

```
Constituent + Address relationship
```

not the physical Address itself.

This is an important normalization idea:

> **Relationship attributes belong in the relationship table.**

Normalization of Gift and Fund

Your Gift-to-Fund model is another strong real-world example.

Suppose:

EXAMPLE / FLOW

```
Gift 10000 = $1,000
```

and the donor wants:

EXAMPLE / FLOW

```
$700 → Cancer Research Fund

$300 → Patient Assistance Fund
```

Putting one:

EXAMPLE / FLOW

```
FundId
```

directly on `Gift` would be incorrect because one Gift can have multiple Funds.

Putting:

EXAMPLE / FLOW

```
Fund1Id
Fund2Id
```

would violate good 1NF design.

Instead:

EXAMPLE / FLOW

```
Gift
   |
   +---- GiftAllocation → Fund 300 → $700
   |
   +---- GiftAllocation → Fund 301 → $300
```

This is exactly why `GiftAllocation` exists.

Denormalization

Normalization reduces redundancy.

Sometimes a real application intentionally introduces controlled redundancy.

That is called:

EXAMPLE / FLOW

```
denormalization
```

Simple Definition

Denormalization is the intentional storage of redundant or precomputed data to improve a specific workload, usually read or reporting performance.

Fundraising Denormalization Example

Imagine your application frequently displays:

EXAMPLE / FLOW

```
LifetimeGivingTotal
```

for every Constituent.

Normalized approach:

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    SUM(TotalAmount)
FROM fundraising.Gift
GROUP BY ConstituentId;
```

Now suppose there are:

EXAMPLE / FLOW

```
hundreds of millions of Gift rows
```

and this total must be read constantly.

A team might consider adding:

EXAMPLE / FLOW

```
Constituent.LifetimeGivingTotal
```

as a cached/precomputed value.

That would be denormalization.

Why Is This Denormalization?

Because the authoritative data already exists in:

EXAMPLE / FLOW

```
Gift.TotalAmount
```

The lifetime total can be calculated.

Storing it again means the same logical information now exists in:

EXAMPLE / FLOW

```
Gift rows
```

and:

EXAMPLE / FLOW

```
Constituent summary column.
```

Denormalization Risk

Suppose:

EXAMPLE / FLOW

```
SUM(Gift.TotalAmount)
= $50,000
```

but:

EXAMPLE / FLOW

```
Constituent.LifetimeGivingTotal
= $47,000
```

Which one is correct?

Now you have a consistency problem.

Controlled Denormalization

Good denormalization should have:

EXAMPLE / FLOW

```
a measured performance problem,

a clear authoritative source,

a synchronization strategy,

validation,

documentation,

and monitoring.
```

Do not denormalize simply because:

EXAMPLE / FLOW

```
joins look complicated.
```

When to Consider Denormalization

Possible examples:

EXAMPLE / FLOW

```
large reporting workloads,

read-heavy analytics,

expensive repeated aggregations,

materialized summary data,
```

```
data warehouse designs,

high-volume dashboard calculations.
```

When Not to Denormalize

Avoid it when:

EXAMPLE / FLOW

```
there is no measured performance problem,

the normalized query already performs well,

data changes frequently,

consistency is critical,

or duplicated values are difficult to synchronize.
```

Normalization vs Denormalization

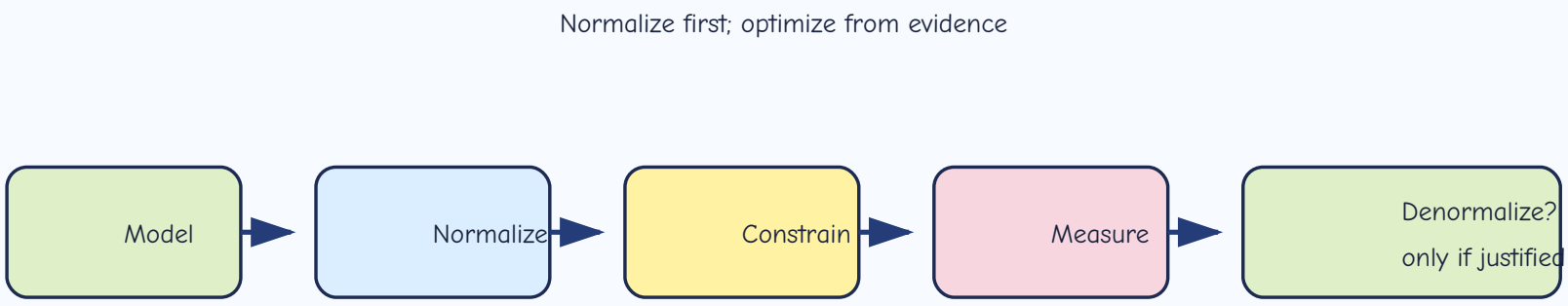
Normalization	Denormalization
Reduce unnecessary redundancy	Intentionally introduce redundancy
Stronger consistency	More synchronization required
More relational separation	May reduce runtime joins/calculations
Great OLTP baseline	Often useful in reporting/analytics
Easier updates	Potentially faster reads
Default design approach	Measured optimization

Normalize First, Optimize Second

A good application-design process is:

DIAGRAM

Normalize First, Optimize Second



EXAMPLE / FLOW

```
Model business correctly
        |
        v
Normalize appropriately
        |
        v
Add correct constraints
        |
        v
Create useful indexes
        |
        v
Measure real workload
        |
        v
Tune queries
        |
        v
Only then consider denormalization
```

Normalization vs Performance

A common myth is:

> Normalization makes databases slow because it creates joins.

This is too simplistic.

Properly indexed normalized tables can perform extremely well.

Performance depends on:

EXAMPLE / FLOW

```
indexes,

query design,

statistics,
```

```
cardinality,

row counts,

data distribution,

hardware,

and workload.
```

Do not destroy integrity just to avoid joins.

Choosing the Appropriate Normal Form

Not every business table needs to be forced mechanically to 5NF.

A practical progression is:

EXAMPLE / FLOW

```
1NF
→ fundamental requirement

2NF
→ important where composite keys exist

3NF
→ strong normal OLTP target

BCNF
→ check unusual determinant/key relationships

4NF
→ check independent multivalued facts

5NF
→ specialized multi-relation join dependencies
```

Typical Fundraising OLTP Target

For an application like:

EXAMPLE / FLOW

```
FundraisingDb
```

a strong practical target is usually:

EXAMPLE / FLOW

```
3NF
```

with tables naturally satisfying:

EXAMPLE / FLOW

```
BCNF
```

where appropriate.

4NF and 5NF are applied when those special dependency patterns actually exist.

Do not create additional tables just so you can claim:

EXAMPLE / FLOW

```
"We reached 5NF."
```

Normalization Is Not "More Tables Is Better"

Bad thinking:

EXAMPLE / FLOW

```
20 tables are more normalized than 10 tables.
```

Incorrect.

Normalization depends on:

EXAMPLE / FLOW

```
business rules

keys

dependencies

and fact ownership.
```

A database with 100 unnecessary lookup tables can still be poorly designed.

Status and Type Columns

Normalization does not mean every:

EXAMPLE / FLOW

```
Status

Type

Priority
```

needs a separate table.

For example:

EXAMPLE / FLOW

```
Task.Priority
```

might simply be a constrained column if values are small and stable.

Create a separate reference table only when the business concept needs:

EXAMPLE / FLOW

```
its own metadata,

management,

relationships,

or lifecycle.
```

Practical Normalization Process

When you receive a large table, use this approach.

Step 1 — Understand What One Row Represents

Ask:

> What business event or entity does one row describe?

Example:

EXAMPLE / FLOW

```
One GiftAllocation row
```

should represent:

> A specific portion of a particular Gift allocated to a particular Fund.

Step 2 — Identify Candidate Keys

Ask:

> What value or combination can uniquely identify this business fact?

Actual implementation:

EXAMPLE / FLOW

```
GiftAllocationId
```

Business combination may also be:

EXAMPLE / FLOW

```
GiftId + FundId
```

if only one allocation per fund per gift is permitted.

Step 3 — Write Dependencies in Plain English

Do not begin with symbols.

Write:

EXAMPLE / FLOW

```
GiftId determines the Gift's date and total amount.

FundId determines the Fund's name.

GiftId together with FundId determines the allocation amount.
```

Then optionally shorten them:

EXAMPLE / FLOW

```
GiftId → GiftDate, TotalAmount

FundId → FundName

(GiftId, FundId) → AllocationAmount
```

Step 4 — Check 1NF

Look for:

EXAMPLE / FLOW

```
lists inside columns,

Fund1/Fund2/Fund3 columns,

repeating groups,

multiple business values packed together.
```

Step 5 — Check 2NF

If a candidate key is composite, ask:

> Does a non-key fact depend on only one part of the key?

Fundraising example:

EXAMPLE / FLOW

```
GiftDate depends only on GiftId.
```

Therefore it should not sit in a GiftId+FundId relationship table.

Step 6 — Check 3NF

Ask:

> Does one non-key fact determine another non-key fact?

Example:

EXAMPLE / FLOW

```
Gift stores ConstituentId.

ConstituentId determines ConstituentName.
```

Therefore ConstituentName belongs in Constituent, not Gift.

Step 7 — Check BCNF

For every dependency ask:

Does the determinant uniquely identify the row?

If not, investigate a possible BCNF violation.

Step 8 — Check Independent Multi-Valued Relationships

Ask:

Does one entity independently have several addresses and several interactions stored in the same table?

If yes:

EXAMPLE / FLOW

```
possible 4NF issue.
```

Step 9 — Check Complex Join Dependencies

For specialized three-way relationships, ask:

Can this relationship be decomposed into smaller relationships without creating false combinations?

That is the 5NF question.

Step 10 — Verify Losslessness

After decomposing:

Can I join the tables and reconstruct exactly the valid original business facts?

If not:

EXAMPLE / FLOW

```
the decomposition is wrong.
```

Step 11 — Check Dependency Preservation

Ask:

Can important business dependencies still be enforced naturally in the new tables?

Three-Frame Method for Normalization Questions

For interview or exam normalization problems, use this method.

EXAMPLE / FLOW

```
FRAME 1
Current table
```

```
        ↓

FRAME 2
Identify keys and dependencies

        ↓

FRAME 3
Move each fact to the correct table
```

Example Three-Frame Method

Frame 1

EXAMPLE / FLOW

```
GiftId
FundId
GiftDate
FundName
AllocationAmount
```

Frame 2

EXAMPLE / FLOW

```
GiftDate
belongs to GiftId.

FundName
belongs to FundId.

AllocationAmount
belongs to GiftId + FundId.
```

Frame 3

EXAMPLE / FLOW

```
Gift
Fund
GiftAllocation
```

This is much easier to understand than memorizing dependency notation without understanding the business meaning.

Common Normalization Mistakes

Mistake 1 — Thinking 1NF Means Fully Normalized

A table can be in 1NF and still contain:

EXAMPLE / FLOW

```
partial dependencies

transitive dependencies

huge redundancy.
```

Mistake 2 — Thinking 2NF Means Two Tables

The number:

EXAMPLE / FLOW

```
2
```

does not mean:

EXAMPLE / FLOW

```
two tables.
```

2NF means:

EXAMPLE / FLOW

```
no partial dependency.
```

Mistake 3 — Thinking 3NF Means Three Tables

Likewise:

EXAMPLE / FLOW

```
3NF
```

has nothing to do with having three tables.

Mistake 4 — Looking Only at Primary Key

Formal normalization considers:

EXAMPLE / FLOW

```
all candidate keys.
```

A dependency problem can involve a candidate key that was not selected as the primary key.

Mistake 5 — Splitting Tables Randomly

More tables do not automatically mean more normalization.

Every decomposition should have a reason based on:

EXAMPLE / FLOW

```
business rules

and dependencies.
```

Mistake 6 — Using Names as Relationship Keys

Bad:

EXAMPLE / FLOW

```
Gift.ConstituentName
```

as the relationship.

Better:

EXAMPLE / FLOW

```
Gift.ConstituentId
```

Names can change and may not be unique.

Mistake 7 — Storing Many Values in One Column

Bad:

EXAMPLE / FLOW

```
Gift.FundIds = '300,301,302'
```

Better:

EXAMPLE / FLOW

```
GiftAllocation
```

one row per Gift/Fund relationship.

Mistake 8 — Confusing Historical Facts with Redundancy

Sometimes two similar-looking values are actually different business facts.

Example:

EXAMPLE / FLOW

```
Fund.CurrentName
```

and a historical report snapshot of a Fund name could intentionally differ if the business requires point-in-time reporting.

Likewise, stored historical values are not necessarily normalization mistakes.

Business meaning comes first.

Mistake 9 — Automatically Removing Every Repeated Value

Repeated:

EXAMPLE / FLOW

```
ConstituentId
```

across Gift rows is valid.

It represents:

EXAMPLE / FLOW

```
one Constituent having many Gifts.
```

Do not normalize away legitimate relationships.

Mistake 10 — Denormalizing Before Measuring

Do not duplicate values simply because:

EXAMPLE / FLOW

```
"joins might be slow."
```

Measure first.

Master Normalization Example

Start with the bad table:

EXAMPLE / FLOW

```
GiftDonationDetails
-----------------------------------------------------
GiftId
GiftDate
TotalAmount

ConstituentId
ConstituentName

CampaignId
CampaignName

FundId
FundName

AllocationAmount
```

First Observation

Each row has one Fund allocation.

So it may already appear to satisfy basic 1NF.

2NF Analysis

Assume business key:

EXAMPLE / FLOW

```
GiftId + FundId
```

GiftDate depends only on:

EXAMPLE / FLOW

```
GiftId.
```

TotalAmount depends only on:

EXAMPLE / FLOW

```
GiftId.
```

FundName depends only on:

EXAMPLE / FLOW

```
FundId.
```

Therefore:

EXAMPLE / FLOW

```
partial dependencies exist.
```

Move those facts to:

EXAMPLE / FLOW

```
Gift
Fund
```

3NF Analysis

Inside Gift, suppose we still had:

EXAMPLE / FLOW

```
ConstituentId
ConstituentName

CampaignId
CampaignName
```

ConstituentName is determined by:

EXAMPLE / FLOW

```
ConstituentId.
```

CampaignName is determined by:

EXAMPLE / FLOW

```
CampaignId.
```

Therefore they are transitive from GiftId.

Move them to:

EXAMPLE / FLOW

```
Constituent
Campaign
```

Final Model

EXAMPLE / FLOW

```
Constituent
    |
    | 1:M
    v
Gift
    |
    +----------> Campaign
    |
    | 1:M
    v
GiftAllocation
    |
    | M:1
    v
Fund
```

That is essentially the central normalized fundraising model you already designed.

Complete Normal-Form Memory Table

Form	Ask This Question	Fundraising Example
UNF	Are multiple values packed together?	Several Fund allocations in one Gift column
1NF	Are repeating values stored as rows?	One GiftAllocation row per allocation
2NF	Does a fact depend on only part of a composite key?	GiftDate only depends on GiftId; FundName only on FundId
3NF	Does one non-key fact determine another?	ConstituentId determines ConstituentName
BCNF	Is every determinant a superkey?	Special overlapping-key fundraiser/campaign exercise
4NF	Are independent multi-valued facts mixed?	Constituent addresses + interactions
5NF	Can complex relationships be decomposed without false joins?	Constituent/Campaign/Fund relationship analysis

Dependency Memory Table

Dependency	Plain-English Meaning	Fundraising Example
Functional	One value determines another	FundId determines FundName
Full functional	Whole composite key is required	GiftId + FundId determine AllocationAmount
Partial	Only part of composite key is required	GiftId alone determines GiftDate
Transitive	Key determines A, A determines B	GiftId → ConstituentId → DisplayName
Multivalued	One key independently has many values	Constituent has many Addresses
Join dependency	Complex relationship may be reconstructable from smaller relationships	Advanced Constituent/Campaign/Fund example

25 Interview Questions

1. What is database normalization?

Database normalization is the process of organizing relational data so that each business fact is stored in an appropriate table, unnecessary redundancy is reduced, and insert, update, and delete anomalies are prevented.

In `FundraisingDb`, constituent details live in `Constituent`, gift facts in `Gift`, fund details in `Fund`, and the Gift-to-Fund relationship in `GiftAllocation`.

2. Why is normalization important?

Normalization improves consistency and integrity.

For example, storing John Smith's name once in `Constituent` is safer than repeating the name in every Gift row. A change then requires one update instead of hundreds.

3. What is data redundancy?

Data redundancy means unnecessarily storing the same business fact multiple times.

Repeating:

EXAMPLE / FLOW

```
ConstituentName = John Smith
```

in every Gift row would be redundant.

Repeating `ConstituentId` as a foreign key is not a problem because it represents the real one-to-many relationship.

4. What is an update anomaly?

An update anomaly occurs when the same fact exists in several rows and all copies must be changed.

If a donor name were repeated in 500 Gift rows, renaming the donor would require 500 updates and could leave inconsistent values.

5. What is an insert anomaly?

An insert anomaly occurs when a business fact cannot be stored without creating an unrelated fact.

For example, if Campaign information existed only inside Gift rows, you could not create a new Campaign until it received its first Gift.

6. What is a delete anomaly?

A delete anomaly occurs when deleting one business fact accidentally removes another.

If Fund information existed only in GiftAllocation rows, deleting the last allocation to a Fund could accidentally remove the only stored information describing that Fund.

7. What is a functional dependency?

A functional dependency means one value determines another value according to business rules.

For example:

EXAMPLE / FLOW

```
FundId determines FundName.
```

Once you know the FundId, the corresponding Fund row determines its name.

The compact notation is:

EXAMPLE / FLOW

```
FundId → FundName
```

8. What is a full functional dependency?

A full functional dependency requires the complete composite key.

For example, if one allocation is uniquely identified by:

EXAMPLE / FLOW

```
GiftId + FundId
```

then `AllocationAmount` requires both values.

GiftId alone is insufficient, and FundId alone is insufficient.

9. What is a partial functional dependency?

A partial dependency occurs when a non-key fact depends on only one part of a composite key.

In a bad `(GiftId, FundId)` table:

EXAMPLE / FLOW

```
GiftDate
```

depends only on GiftId.

EXAMPLE / FLOW

```
FundName
```

depends only on FundId.

Those are partial dependencies.

10. What is a transitive dependency?

A transitive dependency occurs when the key determines one non-key value, which then determines another value.

Example:

EXAMPLE / FLOW

```
GiftId identifies ConstituentId.

ConstituentId identifies DisplayName.
```

Therefore GiftId determines DisplayName only indirectly.

DisplayName belongs in Constituent, not Gift.

11. What is First Normal Form?

1NF requires a proper relational structure without repeating groups or multiple independent business values packed into one field.

For example, don't store:

EXAMPLE / FLOW

```
Gift.FundIds = '300,301'
```

Use separate `GiftAllocation` rows instead.

12. What is Second Normal Form?

2NF requires 1NF and removes partial dependencies.

Every non-key fact should depend on the complete candidate key.

For a Gift/Fund allocation relationship, `AllocationAmount` belongs to the relationship, while `GiftDate` belongs to Gift and `FundName` belongs to Fund.

13. Why is 2NF mainly associated with composite keys?

A partial dependency means depending on only part of a key.

A single-column key has no smaller key part.

Therefore the classic 2NF partial-dependency problem requires a composite candidate key.

14. What is Third Normal Form?

3NF requires 2NF and removes inappropriate transitive dependencies.

Non-key facts should not depend on other non-key facts when those values actually belong to another entity.

For example, Gift should store `ConstituentId`, not repeat `ConstituentName`.

15. What is the difference between 1NF, 2NF, and 3NF?

EXAMPLE / FLOW

```
1NF
→ Remove repeating groups.

2NF
→ Remove partial dependencies.

3NF
→ Remove transitive dependencies.
```

In FundraisingDb:

EXAMPLE / FLOW

```
1NF
→ Gift allocations become rows.

2NF
→ Gift facts and Fund facts move out of allocation relationship.

3NF
→ Constituent and Campaign descriptive facts move out of Gift.
```

16. What is BCNF?

BCNF is a stricter form of 3NF.

For every non-trivial dependency, the determinant should be a superkey.

In simple language:

> Anything that determines another fact should itself uniquely identify the row.

17. What is the difference between 3NF and BCNF?

3NF allows some special dependencies when the dependent attribute is part of a candidate key.

BCNF is stricter:

EXAMPLE / FLOW

```
every determinant must be a superkey.
```

The difference mainly appears in unusual designs involving multiple overlapping candidate keys.

18. What is a multivalued dependency?

A multivalued dependency occurs when one entity has an independent collection of multiple values.

For example, one Constituent can independently have many Addresses and many Interactions.

Combining both collections into one table creates unnecessary cross-product rows.

19. What is Fourth Normal Form?

4NF separates independent multivalued relationships.

Your FundraisingDb naturally follows this by storing:

EXAMPLE / FLOW

```
ConstituentAddress
```

separately from:

EXAMPLE / FLOW

```
Interaction.
```

Addresses and interactions are independent child collections of Constituent.

20. What is Fifth Normal Form?

5NF handles complex join dependencies.

It asks whether a multi-way relationship can be decomposed into smaller relationships and later reconstructed exactly without generating false business combinations.

It is far less common than 1NF–3NF in normal application development.

21. What is lossless decomposition?

Lossless decomposition means splitting one table into several tables while retaining the ability to join them back together and reconstruct exactly the valid original information.

Separating Gift and Constituent is lossless because `Gift.ConstituentId` preserves the relationship.

22. What is lossy decomposition?

Lossy decomposition occurs when joining the decomposed tables either loses information or produces false rows.

For example, removing the direct Gift-to-Fund relationship and attempting to reconstruct allocations only through Campaign could create Gift/Fund combinations that never actually occurred.

23. What is dependency preservation?

Dependency preservation means important original business dependencies can still be enforced naturally after decomposition.

For example:

EXAMPLE / FLOW

```
FundId determines FundName
```

is preserved because both values remain together in `fundraising.Fund`.

24. What is denormalization?

Denormalization is the intentional introduction of redundant or precomputed data for a measured performance reason.

For example, storing a precomputed donor lifetime-giving total on `Constituent` would be denormalization because the total can already be calculated from `Gift`.

Such data must be carefully synchronized.

25. Which normal form should a SQL Server business application use?

There is no requirement to force every table to 5NF.

For a transactional OLTP system such as `FundraisingDb`, 3NF is a strong practical baseline, with BCNF, 4NF, and 5NF applied when their specific dependency problems actually exist.

The goal is not:

EXAMPLE / FLOW

```
highest possible normal form.
```

The goal is:

EXAMPLE / FLOW

```
correct business modeling
+
data integrity
+
lossless relationships
+
maintainability
+
appropriate performance.
```

Final Revision Diagram

EXAMPLE / FLOW

```
RAW FUNDRAISING DATA
        |
        v
UNF
Several funds packed into one Gift row
        |
        | Separate repeating values
        v
1NF
One allocation per row
        |
        | Remove partial dependencies
        v
2NF
Gift facts → Gift
Fund facts → Fund
Allocation fact → GiftAllocation
        |
        | Remove transitive dependencies
        v
3NF
Constituent facts → Constituent
```

EXAMPLE / FLOW (CONT. 2)

```
Campaign facts → Campaign
Gift keeps only related IDs
        |
        | Check unusual determinants
        v
BCNF
```

```
Every determinant should be a superkey
        |
        | Separate independent many-valued facts
        v
4NF
Addresses separate from Interactions
        |
        | Analyze complex join dependencies
        v
5NF
No unnecessary decomposable multi-way relationship
```

Final Easy Memory Rule

EXAMPLE / FLOW

```
1NF
→ Fix repeating values.

2NF
→ Make every fact depend on the whole composite key.

3NF
→ Move facts that really belong to another entity.

BCNF
→ Every determinant should uniquely identify a row.

4NF
→ Separate independent collections.

5NF
→ Do not keep unnecessary complex join dependencies.
```

The single most important normalization question is:

> **What business thing does this fact actually describe?**

If:

EXAMPLE / FLOW

```
GiftDate
```

describes Gift, keep it in Gift.

If:

EXAMPLE / FLOW

```
FundName
```

describes Fund, keep it in Fund.

If:

EXAMPLE / FLOW

```
AllocationAmount
```

describes the relationship between Gift and Fund, keep it in GiftAllocation.

If:

EXAMPLE / FLOW

```
DisplayName
```

describes Constituent, keep it in Constituent.

Once you consistently answer that question, normalization becomes much easier to understand than simply memorizing `A → B` formulas.